

Programming Methodology - SE1012

Lab 06 – Selection Structures, Repetition and Loop Statements



Access Lab 6 via GitHub:

<https://classroom.github.com/a/okOCscjC>

Outline of Today's Lab



- If – else Statement
- Switch Statement
- Question 1: GPA Decision Table
- Question 2: BMI Calculator
- Question 3: Counting Loops with `while` Statement
- Question 4: Using the `for` Statement
- Question 5: Using `do-while` for Input Validation
- Question 6: Loop Control with `break` and `continue`
- Question 7: Avoiding Infinite Loops

If – else Statement

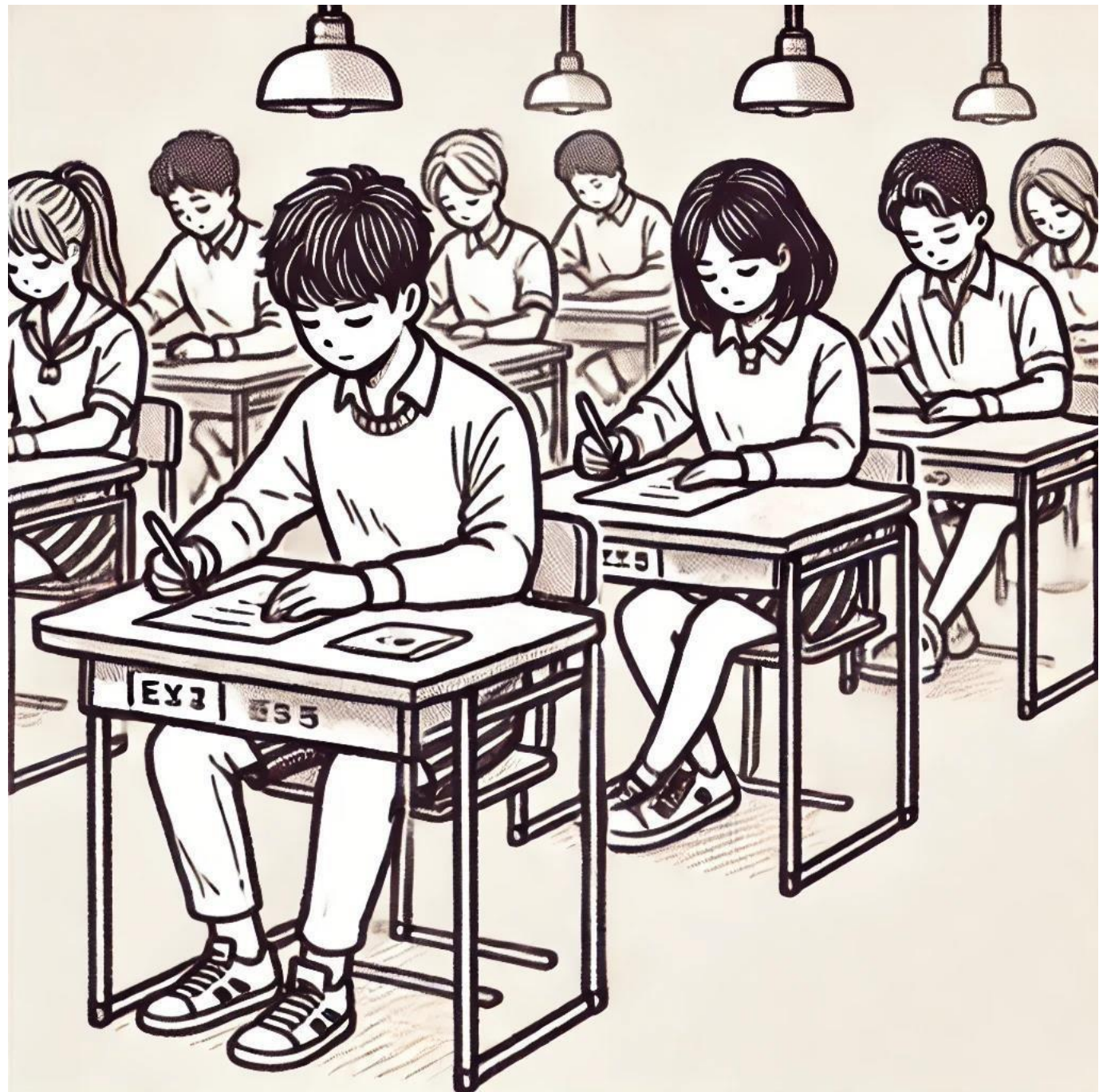
```
if (condition) {  
    // Code to execute if condition is true  
} else {  
    // Code to execute if condition is false  
}
```

Switch Statement

```
switch (variable) {  
    case value1:  
        // Code to execute if variable == value1  
        break;  
    case value2:  
        // Code to execute if variable == value2  
        break;  
    // Other cases  
    default:  
        // Code to execute if variable doesn't match any case  
}  

```

Question 1: GPA Decision Table



GPA Decision Table

Write a C program to implement the following decision table using a nested `if` statement. Assume that the grade point average (GPA) is within the range 0.0 through 4.0.

Grade Point Average	Transcript Message
0.0–0.99	Failed semester—registration suspended
1.0–1.99	On probation for next semester
2.0–2.99	(no message)
3.0–3.49	Dean's list for semester
3.5–4.00	Highest honors for semester

Question 2: BMI Calculator

BODY MASS INDEX



BMI Calculator

Write a program that calculates the user's Body Mass Index (BMI) and categorizes it as underweight, normal, overweight, or obese, based on the following table from the United States Centers for Disease Control:

BMI Category	Range
Underweight	$\text{BMI} < 18.5$
Normal weight	$18.5 \leq \text{BMI} < 25$
Overweight	$25 \leq \text{BMI} < 30$
Obese	$\text{BMI} \geq 30$

To calculate BMI based on weight in pounds (`wt_lb`) and height in inches (`ht_in`), use this formula (rounded to tenths):

```
BMI = (wt_lb / (ht_in * ht_in)) * 703;
```



Introduction to Looping Constructs

- **while Loop:** Repeats a block of code as long as the condition is true
- **do-while Loop:** Similar to while but ensures the block executes at least once
- **for Loop:** Used when the number of iterations is known beforehand

```
while (condition) {  
    // Code to execute as long as the condition is true  
}
```

```
do {  
    // Code to execute at least once  
} while (condition);
```

```
for (initialization; condition; update) {  
    // Code to execute repeatedly  
}
```

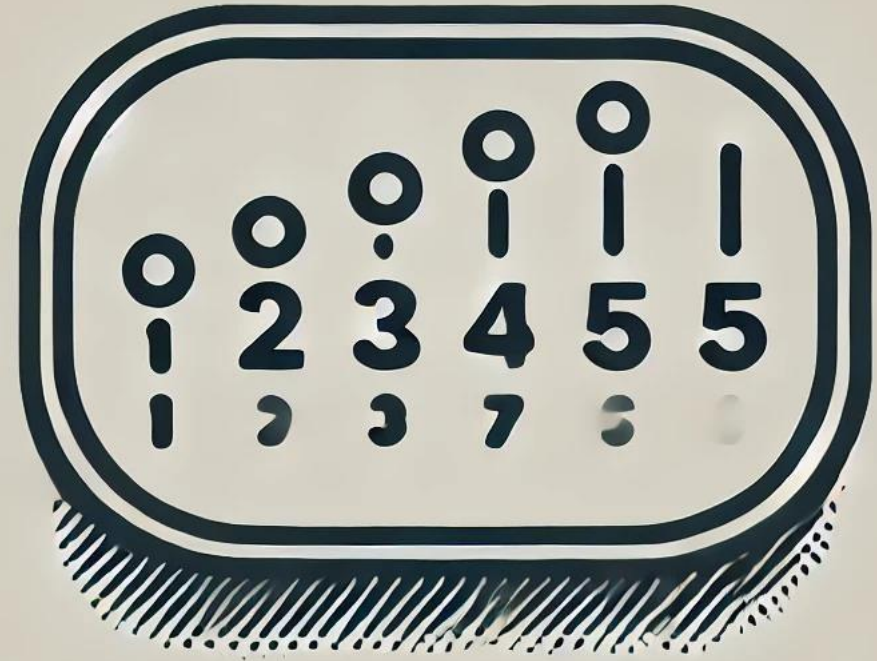
Managing Loop Flow

- **break Statement:** Exits the loop immediately, regardless of the condition
- **continue Statement:** Skips the remaining code in the current iteration and moves to the next iteration

```
if (condition) {  
    break; // Exit the loop  
}
```

```
if (condition) {  
    continue; // Skip to the next iteration  
}
```

Question 3: Counting Loops with `while` Statement



Counting Loops with **while** Statement

- **Scenario & Task:**

Imagine you're building a simple pedometer that counts the number of steps a user takes. To simulate this, write a program using a `while` loop to count from 1 to 5, printing each number on a new line, as if you're counting the steps taken.

- **Test Case:**

Expected output:

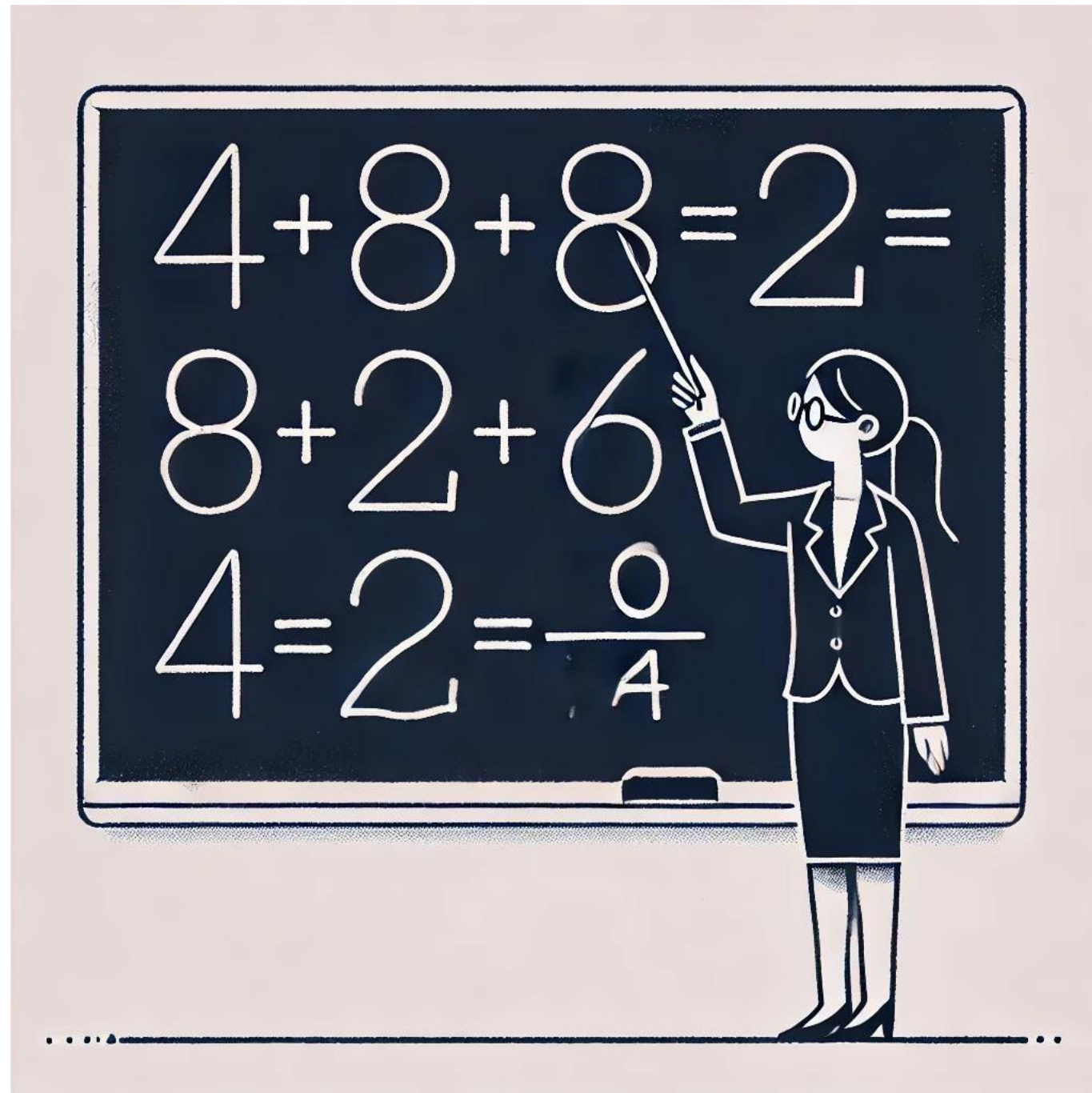
```
1
2
3
4
5
```



- **Hint:**

Initialize a counter variable at 1, then increment it by 1 in each iteration of the loop until it reaches 5.

Question 4: Using the for Statement



Using the **for** Statement

- **Scenario & Task:**

You're helping a teacher with a program to display the first few multiples of a number for a classroom exercise. Write a `for` loop to print the first 5 multiples of 4, which could be used to explain multiplication to students.

- **Test Case:**

Expected output:

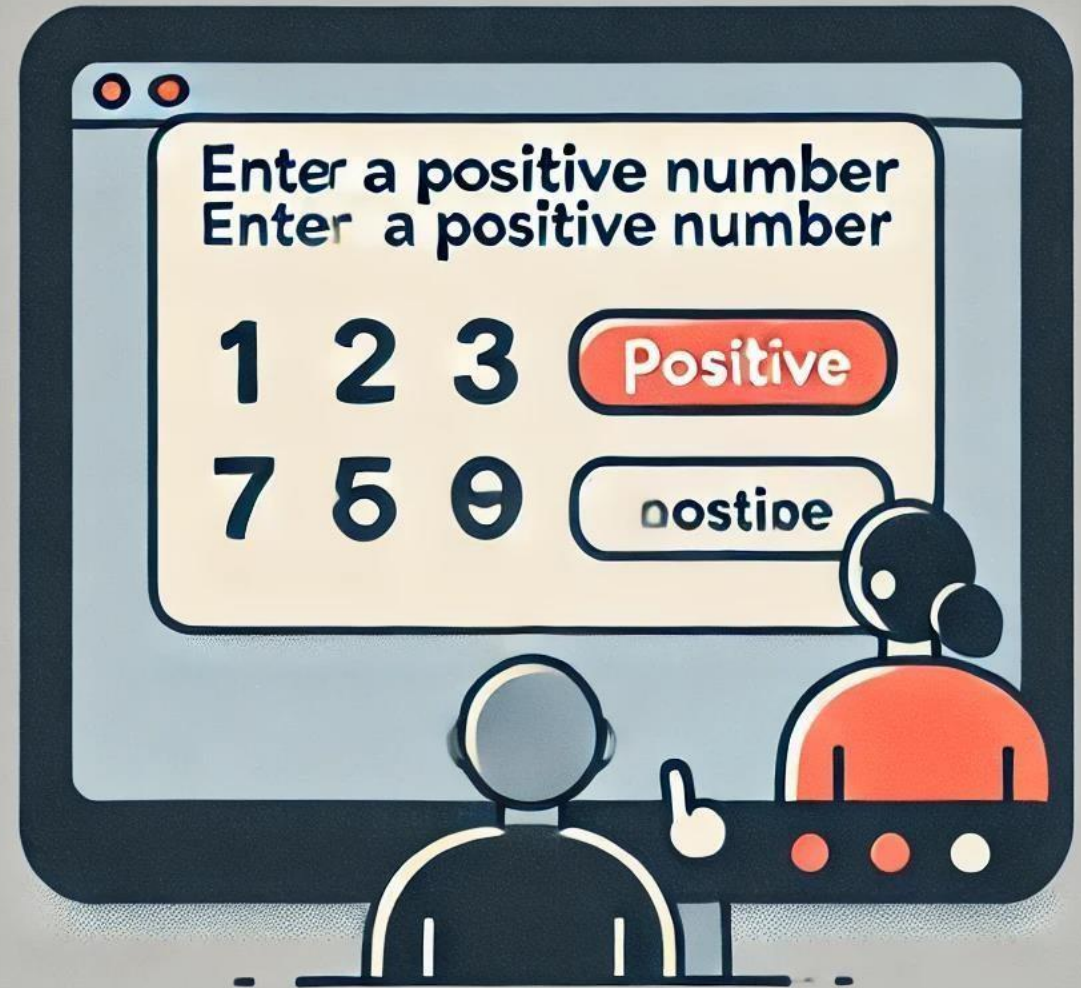
```
4
8
12
16
20
```



- **Hint:**

Start with 4 and multiply it by the loop index (starting from 1 up to 5) in each iteration.

Question 5: Using do-while for Input Validation



Using **do-while** for Input Validation

- **Scenario & Task:**

In an application where users need to enter positive numbers to proceed, ensure that negative inputs are rejected, and the user is prompted until a valid input is provided. Write a `do-while` loop that asks the user for a positive number. If a negative number is entered, prompt the user again until a positive number is provided.

- **Test Case:**

If the user inputs: `-1`, `-5`, `3`, the expected output:

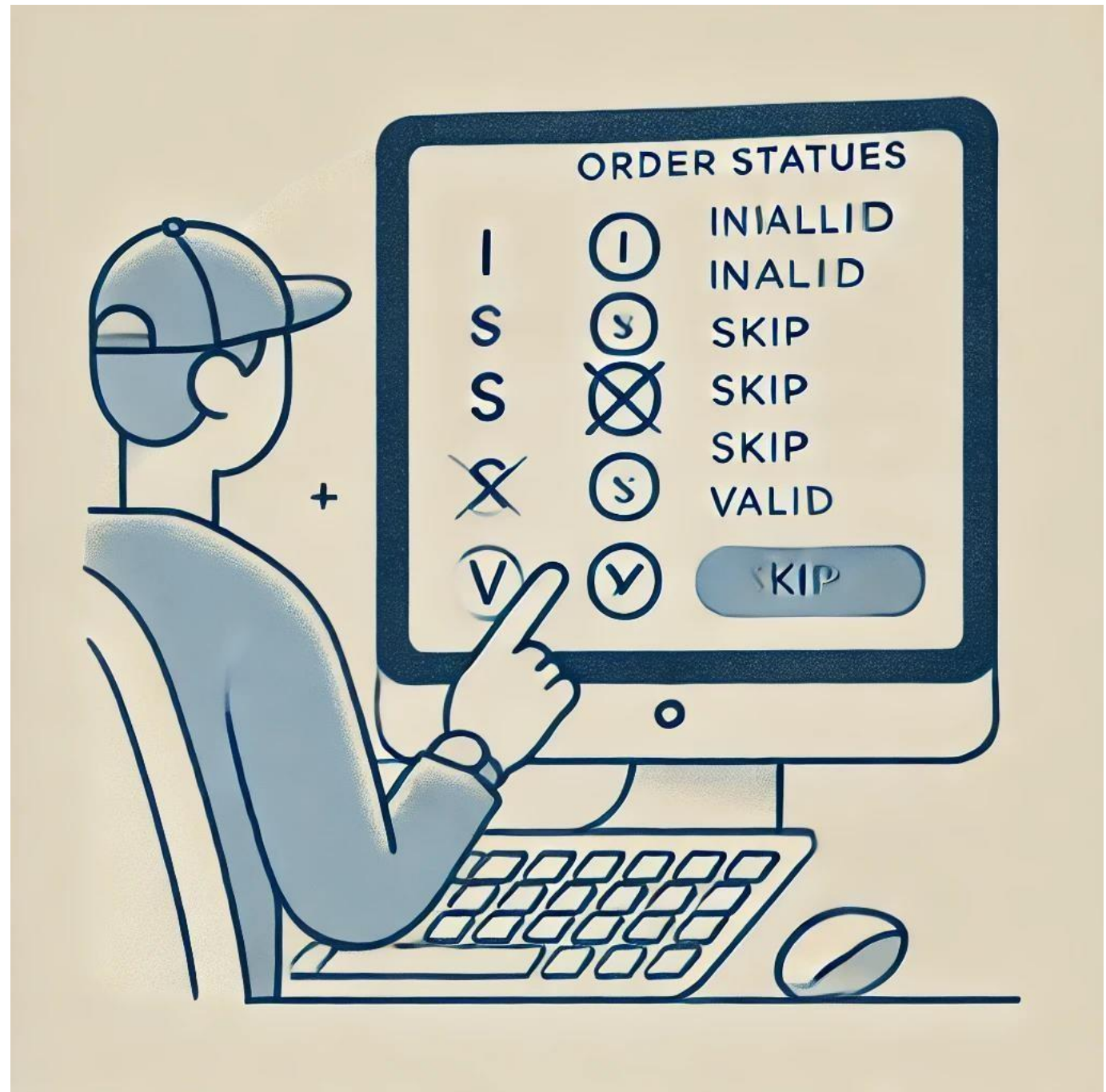
```
Please enter a positive number:  
Please enter a positive number:  
You entered: 3
```



- **Hint:**

The `do-while` loop should ensure the user is asked at least once and continues until a positive number is entered.

Question 6: Loop Control with break and continue



Loop Control with **break** and **continue**

◦ Scenario & Task:

You are developing a system that processes customer orders. The system should skip over certain invalid orders, continue processing others, and stop processing entirely if an error is encountered. Write a program that prompts the user to enter the status of 5 orders using the following codes:

- "I" for Invalid (skip this order)
- "S" for Skip (stop processing further orders)
- "V" for Valid (process this order)

The program should use a `for` loop to process these orders. If the order is marked as "Invalid," use the `continue` statement to skip processing. If the order is marked as "Skip," use the `break` statement to stop processing further orders. Otherwise, process the valid orders.

◦ Test Case:

If the user inputs the following statuses: `V`, `V`, `I`, `S`, `V`, the expected output should be:

```
Processing order 1
Processing order 2
Skipping order 3
Stopping at order 4
```

◦ Hint:

Use `continue` to skip the "Invalid" orders and `break` to stop processing when the "Skip" code is encountered.

Question 7: Avoiding Infinite Loops



Avoiding Infinite Loops

- **Scenario & Task:**

Imagine you're designing a software timer that counts down from a specified time. To avoid infinite loops, ensure the countdown stops when it reaches zero. Write a `while` loop that counts down from 5 to 0, ensuring that the loop exits correctly when the countdown reaches zero.

- **Test Case:**

Expected output:

```
5
4
3
2
1
0
Countdown complete.
```



- **Hint:**

Ensure the loop condition allows the loop to exit once the countdown reaches zero.

Lab Submission Guidelines

- Compile a **small report including screenshots showing you work**
- **Rename the report** with **your IT number** and **covert it to a pdf** file (Example ITxxxxxxxxx.pdf)
- Upload the pdf file to the GitHub Classroom

